

Build and Operate Foundry Agents

One-day hosted-agent workshop
Build one agent. Change a specialist team. Host it in Foundry.

DURATION One-day track Timing subject to rehearsal	DELIVERY Guided, notebook-first labs Python 3.14 dev container	TECHNICAL DEPTH Python development Foundations to orchestration
--	--	---

WORKSHOP DESCRIPTION

Build the Healthcare Marketplace Concierge with Microsoft Foundry hosted agents and Microsoft Agent Framework (MAF). Work through a cumulative, synthetic-data scenario: establish identity and models, test and deploy a typed-tool agent, then change and host a multi-agent team that pauses for human review.

OBJECTIVES AND EXPECTED OUTCOMES

- 1

Establish a verified foundation
Create an attendee project in an approved account; verify chat inference.
- 2

Build and deploy real behavior
Add a typed tool, test policy boundaries and invoke an actual hosted Responses version.
- 3

Change a specialist team
Inspect parallel work, structured merge and compliance review; add model-based task routing.
- 4

Review and host the result
Explicitly approve local and hosted packets; retain source-bound, fixed-version acceptance evidence.

The capstone is a hosted team, not another protocol demo.

MAF supplies orchestration and the human-review boundary. Foundry hosts and versions one service containing code-defined specialists. No Prompt Agents, Search service or Lab 6 prerequisites are required. Deployment is a required, explicit, paid lab action.

AGENDA & TIMING READINESS

Sequence	Module	Participant milestone
Before delivery	Administrator readiness	Approved account, network, roles, hosted enablement, quota and working dev container.
First • TBC	Lab 1 Project and chat inference	Approved attendee project, verified chat inference and persisted configuration.
Next • TBC	Lab 4 One hosted agent	Real local tool/policy acceptance, deployment and selected-version invocation.
Capstone • TBC	Lab 7 Multi-agent team	Changed classifier, specialist packet, explicit approval and hosted team invocation.
Close • TBC	Evidence and cleanup review	Inspect accepted outcomes and review attendee-owned resource cleanup with the facilitator.

TBC = time allocation to be confirmed. The sources specify a one-day route, not approved clock times or lab durations. Rehearsal must confirm pacing, breaks and cloud-readiness waits before scheduling delivery.

From a single agent to a hosted team

Participant outcome: "I changed a team of AI specialists, reviewed its work and hosted it in Foundry."

PARTICIPANTS & READINESS

Who should participate

Engineers building AI-enabled applications, including benefits-marketplace solutions. Participants should be comfortable reading and editing Python, using notebooks, and inspecting JSON and HTTP interactions.

Required workstation and tools

Repository Python 3.14 dev container; VS Code with Dev Containers, Python and Jupyter support. For local delivery, Docker Desktop with Linux containers is required.

Use the container's Azure CLI, Azure Developer CLI (azd), Bash and pinned dependencies. Select `/usr/local/bin/python`. **Alternative:** GitHub Codespaces only where organizational policy and Azure network access permit.

Required Azure environment and access

Administrator-approved tenant, subscription, resource group and existing Foundry account; available chat model and quota; an approved network path from the learner container.

Project/chat-model creation rights, hosted-project enablement and identity access must be confirmed. Deployment requires **Foundry Project Manager**; invocation requires **Foundry Agent Consumer or Foundry User**. Model access is separate from deployment and invocation permission. Use Entra authentication.

Safety and governance boundaries

Synthetic data only. No plan ranking, recommendation or medical advice. Keep facts grounded, minimize data and hand off judgment to a licensed advisor. Approval is an explicit learner simulation, not production authorization. Never commit credentials or runtime evidence. Review costs, scope and cleanup; do not bypass network controls.

Included

Three guided notebooks, scaffolded hosted products, synthetic fixtures, typed-tool/policy exercises, MAF workflow/classifier exercise, explicit human review and acceptance checkpoints.

THREE CUMULATIVE LABS, IN ORDER

01 Project, identity and chat inference

Authenticate, confirm approved account scope and create or resolve the attendee project. Select a chat model, review capacity and charges, deploy it and verify real inference. Save the verified configuration and scoped handoff.

This combines project setup and model verification into one foundational milestone: your environment works. No embedding model is provisioned and no separate Lab 2 is needed. Hosted-project enablement and identity access still require administrator confirmation.

04 Build, test, deploy and invoke

Complete the folded local tool and instruction exercises before deployment. Confirm sponsor facts and the enrollment-window refusal, then invoke the selected hosted version. No separate Lab 3 is needed.

07 Build and host a multi-agent team

Inspect the scaffolded MAF graph, route to marketplace/account specialists, merge and compliance-check results. Add a structured classifier, explicitly approve a mixed-case packet, then deploy and review the same team in Foundry.

Delivery methodology

Teach → inspect → demonstrate → edit → verify → hand off. Run each notebook in a fresh kernel and restore validated predecessor evidence instead of repeating provisioning. Learners edit isolated one-day products. Model calls and cloud deployment are explicit actions; a successful local test is not proof of Azure acceptance.

Shared root configuration, port 8088 and quota mean tracks must not run concurrently. File-backed pending state is single-instance, not cross-replica recovery.

Out of scope / not prerequisites

Embedding models, RAG, Prompt Agents, Search/Foundry IQ setup, Blob history, telemetry/evaluation infrastructure, batch Invocations, restart recovery, hosted delegation and production authorization. No production-readiness certification.